



Računarski fakultet

MASTER RAD

TT-GS - Rasterizacija 3D
Gaussian Splatting algoritma
na Tenstorrent hardveru

Vanja Kovinić

M 11/2024

Mentor:

dr Nemanja Ilić

Komisija:

dr Nemanja Ilić

dr Nevena Marić

dr Goran Rakočević

Beograd, jul 2026.

Sadržaj

1	Uvod	2
1.1	Motivacija	2
1.2	Doprinosi i struktura rada	3
2	3D Gaussian Splatting	5
2.1	Reprezentacija scene	6
2.2	Geometrija Gausijana: skala i rotacija	8
2.3	Boja: sferični harmonici	9
2.4	Projekcija na ravan slike	10
2.5	<i>Alpha blending</i>	12
2.6	Tile rasterizacija i kompletan algoritam	13

1 Uvod

1.1 Motivacija

Fotografija prikazuje scenu iz tačno jedne perspektive, dok posmatrač po pravilu želi upravo suprotno - slobodu da se kroz tu istu scenu kreće. Premošćivanje tog jaza, odnosno rekonstrukcija 3d prostora iz skupa fotografija, tako da kasnije scenu možemo posmatrati iz proizvoljnog ugla, jedan je od temeljnih problema računarske grafike/računarskog vida i naziva se *novel view synthesis*. Decenijama je ovaj zadatak počivao na eksplicitnim geometrijskim reprezentacijama (*polygonal meshes* i *point clouds*) koje se teško nose sa složenim materijalima (*materials*), providnošću i sitnim detaljima poput dlake ili lišća.

Prekretnicu je donela **NeRF** [1] metoda (engl. *neural radiance fields*), koja prizor predstavlja *implicitno*, kao neprekidnu funkciju koju aproksimira višeslojni perceptron. Ta funkcija svakoj tački u prostoru i svakom pravcu gledanja pridružuje boju i gustinu, a slika se formira integraljenjem duž zraka koji prolaze kroz scenu (engl. *volume rendering*). **NeRF** postiže fotorealistične rezultate, ali po visokoj ceni: renderovanje jednog piksela zahteva stotine evaluacija neuralne mreže duž odgovarajućeg zraka, zbog čega je izvorna metoda daleko od interaktivne brzine, dok se i sam postupak treninga meri u satima, a radi se po sceni.

3D Gaussian Splatting [2] (**3DGS**) rešava upravo ovaj problem brzine tako što napušta implicitnu reprezentaciju i prizor opisuje eksplicitno - skupom od više miliona anizotropnih 3D Gausovih raspodela ¹ (u nastavku: *Gausijana*), od kojih svaka nosi svoj položaj, kovarijansu, boju i providnost. Umesto skupe obrade zrakova, slika se dobija *rasterizacijom*: Gausijani se projektuju na ravan slike, sortiraju po dubini i potom stapaju u konačnu boju piksela *alpha blending* algoritmom. Time se dobijaju interaktivne brzine (preko 30 fps) uz kvalitet uporediv sa najboljim varijantama NeRF-a, što je 3DGS učinilo dominantnom metodom u oblasti.

¹Anizotropne Gausove raspodele su Gausove raspodele koje imaju različite varijanse za različite pravce.

Ono što je zajedničko i NeRF-u i, posebno, 3DGS-u jeste da su osmišljeni i implementirani za GPU kartice (**CUDA** programski jezik), uz lakše varijante namenjene vebu (**WebGL/WebGPU**). Sam postupak renderovanja čvrsto je vezan za **SIMT** (engl. *Single Instruction, Multiple Threads*) model izvršavanja grafičkih procesora, u kome veliki broj niti sinhrono izvršava isti program nad različitim podacima. U međuvremenu se pojavila nova klasa čipova - akceleratora namenjenih veštačkoj inteligenciji (u nastavku: *ML čipovi*) - koji su optimizovani za izvršavanje modela mašinskog učenja.

Tenstorrentov *Blackhole* čip jedan je takav čip. Kako je reč o novom hardveru, tek počinju da se pojavljuju radovi koji ispituju njegove performanse na različitim zadacima [3, 4, 5], dok renderovanje nije među njihovim primarnim namenama.

Konkretno, postavlja se pitanje: može li se računski najzahtevniji deo 3DGS renderovanja efikasno izvršiti na ML čipu i da li je takvo rešenje konkurentno postojećim? Rad se ograničava na prolaz unapred (engl. *forward pass*), odnosno na renderovanje već istrenirane scene, bez podrške za diferencijabilnu rasterizaciju i treniranje scene od nule.

1.2 Doprinosi i struktura rada

Doprinosi ovog rada mogu se sažeti u nekoliko tačaka:

- **Kernel za *alpha blending* na Tenstorrent hardveru.**

Implementiran je **tt-metal** kernel koje izvršava *alpha blending* deo 3DGS algoritma na Tenstorrent hardveru. To je prva implementacija 3DGS algoritma na Tenstorrent čipu.

- **Poređenje 3 različita *backend-a***

Poređenje performansi 3 različitih *backend-a*: CPU, GPU i Tenstorrent.

- ***Longest-Processing-Time first* (LPT) raspoređivanje zadataka.**

Budući da je broj Gausijana po *tile*-u izrazito neravnomeran, naivna podela *tile*-ova po jezgrima dovodi do toga da ukupno vreme renderovanja diktira najopterećenije jezgro. Umesto toga, implementacija je zasnovana na pravilu najdužeg posla (engl. *Longest-Processing-Time first*), koja osetno ujednačava opterećenje jezgara.

Zajedno, ova rešenja dovode do ubrzanja od preko $20\times$ u odnosu na CPU implementaciju.

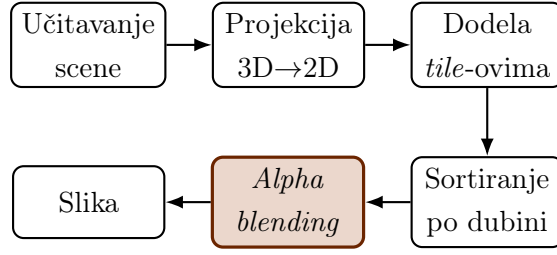
2 3D Gaussian Splatting

3DGS scena je predstavljena *eksplicitno* pomoću stotine hiljada do nekoliko miliona Gausijana raspoređenih u prostoru, od kojih svaki pokriva relativno mali, ograničen deo zapremine. Za razliku od **NeRF**-a, gde je scena skrivena u težinama neuralne mreže, ovde su svi parametri scene direktno dostupni, pa se slika dobija rasterizacijom umesto integraljenjem po zracima, gde se po jednom zraku neuronska mreža više puta koristi za aproksimaciju boje i gustine. Ta razlika je ono što 3DGS čini dovoljno brzim za interaktivno renderovanje.



Slika 1: Primer scene renderovane 3DGS algoritmom na Tenstorrent *Black-hole* čipu. Proviđne, izdužene elipse su karakteristične za reprezentaciju zasnovanu na Gausijanima.

Renderovanje jedne slike svodi se na algoritam prikazan na *Slici 2*: scena se učitava, svaki Gausijan se projektuje na 2D ravan slike, dodeljuje se *tile*-ovima koje pokriva, Gausijani se sortiraju po dubini i, na kraju, stapaju u boju piksela. Poglavlje prati taj redosled - od reprezentacije scene (2.1) do *alpha blending* jednačine (2.5), koja je ujedno i deo kernela koji je prebačen na Tenstorrent čip.



Slika 2: Algoritam 3DGS renderovanja po fazama. Prve četiri faze pripremaju i organizuju podatke; *alpha blending* (istaknuto) je računski najzahtevnija faza i ona je prebačena na Tenstorrent čip.

2.1 Reprezentacija scene

Scena je skup od N Gausijana. Svaki Gausijan opisan je sa četiri grupe parametara: centrom $\mu \in \mathbb{R}^3$, kovarijansom $\Sigma \in \mathbb{R}^{3 \times 3}$, skalarnom vrednošću *opacity* $o \in [0, 1]$ i bojom c , koja je definisana preko sferičnih harmonika (više u poglavlju 2.3). Raspodela gustine Gausijana je definisana Gausovom funkcijom

$$G(x) = \exp\left(-\frac{1}{2}(x - \mu)^\top \Sigma^{-1}(x - \mu)\right), \quad (1)$$

koja ima vrednost 1 u centru i opada ka nuli kako se udaljava od centra. Kovarijansa Σ određuje oblik i rotaciju Gausijana. U *Tabeli 1* je rezime oznaka korišćenih u ovom radu.

Oznaka	Značenje
μ	centar Gausijana (3D)
Σ	3D kovarijansa
S, R	matrica skale i rotacije; $\Sigma = RSS^{\top}R^{\top}$
q	jedinični kvaternion rotacije
o	<i>opacity</i> , $o \in [0, 1]$
c	boja (definisana preko sferičnih harmonika)
SH	sferični harmonici (u radu je korišćen samo stepen 0)
$G(x)$	Gausova funkcija
W, J	<i>view</i> matrica i Jakobijan projekcije
Σ'	2D (projektovana) kovarijansa
α_i	alfa i -tog Gausijana u pikselu ($opacity \times G(x)$)
T_i	transmitansa: $T_i = \prod_{j < i} (1 - \alpha_j)$
C	finalna boja piksela
N, P	broj Gausijana / broj piksela

Table 1: Oznake korišćene u poglavlju.

2.2 Geometrija Gausijana: skala i rotacija

Da bi kovarijansa opisivala stvaran elipsoid, mora biti simetrična i pozitivno semidefinitna; u suprotnom Σ^{-1} iz jednačine (1) ne definiše ispravnu Gausovu raspodelu. Da bi to svojstvo uvek bilo zadovoljeno, Σ se ne pamti direktno, već preko matrica skale i rotacije:

$$\Sigma = R S S^\top R^\top = (RS)(RS)^\top, \quad (2)$$

gde je $S = \text{diag}(s_x, s_y, s_z)$ dijagonalna matrica skale, a R matrica rotacije. Pošto je zapis $(RS)(RS)^\top$ oblika $MM^\top$, dobijena kovarijansa je uvek validna (pozitivno semidefinitna), za bilo koje vrednosti S i R . Geometrijski, jedinična sfera se najpre razvuče pomoću S matrice u elipsoid, a zatim rotira pomoću R matrice (Slika 3).



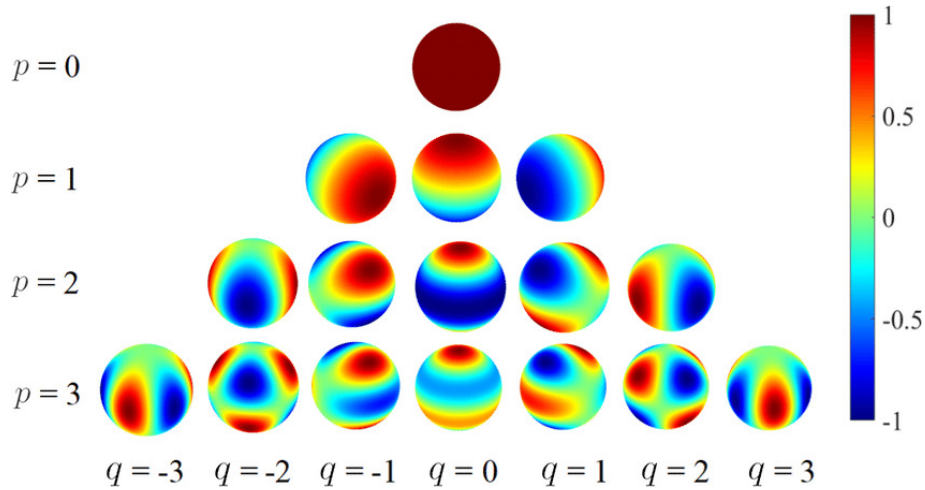
Slika 3: Konstrukcija kovarijanse $\Sigma = RSS^\top R^\top$: jedinična sfera se skalira matricom S , a zatim rotira matricom R . Ovakav zapis uvek daje validnu kovarijansu.

Rotacija se pamti kao jedinični kvaternion $q = (q_0, q_1, q_2, q_3)$ umesto kao matrica, zbog memorijske optimizacije. Od kvaterniona se odgovarajuća matrica rotacije dobija na sledeći način:

$$R(q) = \begin{bmatrix} 1 - 2(q_2^2 + q_3^2) & 2(q_1q_2 - q_0q_3) & 2(q_1q_3 + q_0q_2) \\ 2(q_1q_2 + q_0q_3) & 1 - 2(q_1^2 + q_3^2) & 2(q_2q_3 - q_0q_1) \\ 2(q_1q_3 - q_0q_2) & 2(q_2q_3 + q_0q_1) & 1 - 2(q_1^2 + q_2^2) \end{bmatrix}. \quad (3)$$

2.3 Boja: sferični harmonici

Boja Gausijana zadaje se sferičnim harmonicima (engl. *spherical harmonics*, **SH**) - skupom ortonormiranih baznih funkcija definisanih na jediničnoj sferi. Boja se izražava kao njihova linearna kombinacija, pri čemu koeficijenti određuju kako se ona menja u zavisnosti od pravca gledanja: ista tačka može imati različitu boju posmatrana iz različitih uglova, a viši stepeni **SH** unose detaljnije ugaone varijacije i time omogućavaju reprodukciju refleksija i odsjaja (*Slika 4*). U ovom radu se zbog jednostavnosti implementacije koristi samo multi stepen, čija je jedina bazna funkcija konstanta, pa boja ne zavisi od pravca i svodi se na fiksni RGB vektor $c \in [0, 1]^3$, dat izrazom $c = 0.5 + C_0 \cdot f_{dc}$, gde je $C_0 = 1/(2\sqrt{\pi})$, a f_{dc} multi **SH** koeficijent.



Slika 4: Realni sferični harmonici Y_p^q na sferi, za stepene $p = 0$ do 3 (vrste) i redove $q = -p, \dots, p$ (kolone); boja označava vrednost funkcije (plavo -1 , crveno $+1$). Stepen $p = 0$ (vrh) je konstanta i jedini se koristi u ovom radu, dok viši stepeni uvode sve detaljnije ugaone varijacije [6].

2.4 Projekcija na ravan slike

Da bi se Gausijan renderovao, njegov 3D oblik se mora preslikati u 2D elipsu na ravni slike. Centar se projektuje standardnom *pinhole* projekcijom, ali kovarijansa zahteva više pažnje.

Najpre se Gausijan prebacuje iz koordinata scene u kamera-koordinate matricom W i translacijom t :

$$\mu_{cam} = W\mu + t, \quad \Sigma_{cam} = W\Sigma W^\top. \quad (4)$$

Druga jednakost sledi iz pravila da se kovarijansa pod linearnim preslikavanjem A transformiše kao $\text{cov}(Ax) = A \text{cov}(x) A^\top$: pošto je linearni (rotacioni) deo te transformacije matrica W , kovarijansa se množi sa W i $W^\top$ sa obe strane.

Pinhole projekcija preslikava tačku (x, y, z) iz kamera koordinatnog prostora na ravan slike kao

$$\pi(x, y, z) = \left(f_x \frac{x}{z} + c_x, \quad f_y \frac{y}{z} + c_y \right), \quad (5)$$

gde su f_x, f_y žižne daljine a (c_x, c_y) glavna tačka. Ovo preslikavanje je nelinearno - deli sa z - pa slika Gausove raspodele kroz π u opštem slučaju više nije Gausova.

EWA splatting [7] taj problem rešava aproksimacijom prvog reda: π se u okolini centra zamenjuje svojom tangentnom linearnom aproksimacijom, čija je matrica prvih izvoda Jakobijan $J = \partial\pi/\partial(x, y, z)$. Diferenciranjem (5) dobija se

$$J = \begin{bmatrix} f_x/z & 0 & -f_x x/z^2 \\ 0 & f_y/z & -f_y y/z^2 \end{bmatrix}, \quad (6)$$

što je dobra aproksimacija dokle god je Gausijan mali u odnosu na udaljenost od kamere; za Gausijane vrlo blizu kamere linearizacija se kviri, što se rešava odbacivanjem Gausijana čiji poluprečnik pređe određeni prag. Primenom istog pravila o transformaciji kovarijanse, sada sa kompozicijom JW , dobija se 2D kovarijansa

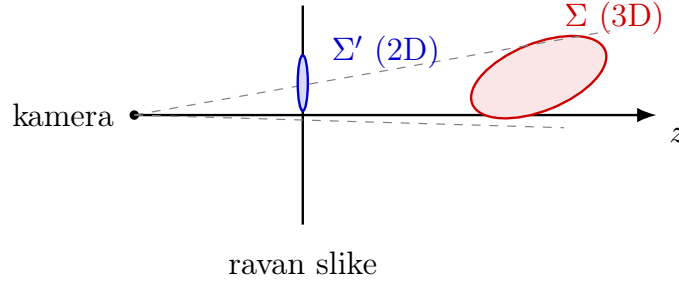
$$\Sigma' = JW\Sigma W^\top J^\top, \quad (7)$$

Pošto je J dimenzije 2×3 , proizvod je direktno 2×2 matrica - kovarijansa elipse na ekranu.

Na dijagonalu Σ' dodaje se mali član (*low-pass filter*, reda 0.3 piksela): bez njega bi Gausijani manji od piksela imali gotovo singularnu Σ' i izazivali *aliasing*, dok proširenje od 0.3 piksela obezbeđuje da svaki pokriva bar oko jednog piksela. Najzad, iz Σ' se računa poluprečnik omotača. Za simetričnu matricu $\Sigma' = \begin{pmatrix} a & b \\ b & c \end{pmatrix}$ sopstvene vrednosti su

$$\lambda_{1,2} = \frac{a+c}{2} \pm \sqrt{\left(\frac{a-c}{2}\right)^2 + b^2}, \quad (8)$$

a veća od njih, λ_{max} , jednaka je varijansi duž glavne ose elipse. Poluprečnik se uzima na 3σ , tj. $r = 3\sqrt{\lambda_{max}}$, čime je obuhvaćen praktično ceo vidljivi opseg Gausijana. Taj poluprečnik određuje koje piksele i koje *tile*-ove Gausijan dotiče. *Slika 5* prikazuje ceo postupak.



Slika 5: Projekcija Gausijana na ravan slike. 3D oblik Σ preslikava se u 2D elipsu Σ' , pri čemu se nelinearna *pinhole* projekcija linearizuje Jakobijanom oko centra (**EWA** splatting).

2.5 Alpha blending

Boja piksela nastaje slaganjem svih Gausijana koji ga prekrivaju, poređanih od najbližeg ka najdaljem. Polazna tačka je *volume rendering* integral [8], koji opisuje svetlost duž zraka, od bliske tačke t_n do daleke t_f , sa bojom koja se emituje u tački $c(t)$ i gustinom $\sigma(t)$:

$$C = \int_{t_n}^{t_f} T(t) \sigma(t) c(t) dt, \quad T(t) = \exp\left(-\int_{t_n}^t \sigma(s) ds\right). \quad (9)$$

Veličina $T(t)$ je transmitansa - udeo svetlosti koji stiže do tačke t . Diskretizacijom po nizu sortiranih Gausijana integral prelazi u sumu

$$C = \sum_{i=1}^n c_i \alpha_i T_i, \quad T_i = \prod_{j=1}^{i-1} (1 - \alpha_j), \quad (10)$$

što je isti *front-to-back* postupak slaganja poznat iz kompozicije slika [9]. Transmitansa T_i je proizvod propuštanja svetlosti svih Gausijana koji su ispred i -tog; kreće od 1 i monotono opada.

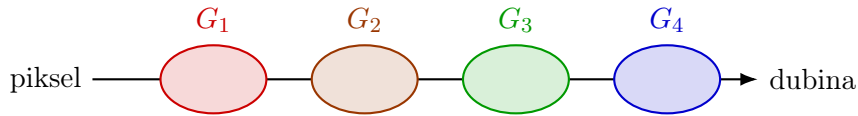
Za 3DGS, alfa vrednost i -tog Gausijana u datom pikselu se računa kao proizvod *opacity* vrednosti sa vrednošću 2D Gausove funkcije u tom pikselu:

$$\alpha_i = o_i \cdot \exp\left(-\frac{1}{2} d^\top \Sigma'^{-1} d\right), \quad d = x_{px} - \mu'_i, \quad (11)$$

gde je d pomeraj piksela od projektovanog centra μ'_i . Slaganje se odvija inkrementalno: za svaki Gausijan ažuriraju se akumulirana boja i transmitansa,

$$C \leftarrow C + c_i \alpha_i T, \quad T \leftarrow T (1 - \alpha_i), \quad (12)$$

a postupak se prekida čim T padne ispod određenog praga, nakon kog dalji Gausijani više ne mogu vidno doprineti pikselu (*Slika 6*). Jednačine (11) i (12) čine srž renderovanja i predstavljaju upravo deo algoritma koji je u ovom radu implementiran kao Tenstorrent kernel.

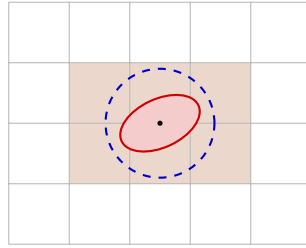


$$T : 1 \rightarrow (1 - \alpha_1) \rightarrow (1 - \alpha_1)(1 - \alpha_2) \rightarrow \dots$$

Slika 6: *Alpha blending* duž zraka jednog piksela. Gausijani poređani po dubini doprinose boji srazmerno svojoj α_i vrednosti i preostaloj transmitansi T , koja monotono opada.

2.6 Tile rasterizacija i kompletan algoritam

Direktna primena jednačine (10) - da svaki piksel prođe kroz sve Gausijane - bila bi $O(N \cdot P)$ za N Gausijana i P piksela, i neupotrebljivo spora na scenama sa milionima Gausijana. Rešenje je prostorna podela: slika se deli na mrežu kvadratnih *tile*-ova, a svaki Gausijan se, na osnovu svog omotača iz Odeljka 2.4, dodeljuje samo *tile*-ovima koje "dodiruje" (Slika 7). Pošto omotač po pravilu pokriva više *tile*-ova, isti Gausijan može se naći u više lista, pa je ukupan broj parova (*Gausijan*, *tile*) veći od N . Pri *alpha blending* renderovanju svaki piksel onda prolazi samo kroz Gausijane svog *tile*-a, kojih je tipično znatno manje nego u celoj sceni.



Slika 7: Dodela *tile*-ovima. Omotač Gausijana (isprekidani krug poluprečnika 3σ) preseca više *tile*-ova (osenčeni), pa se Gausijan upisuje u listu svakog od njih. Pri *alpha blending*-u svaki piksel prolazi samo kroz Gausijane svog *tile*-a.

Da bi se primenila (10), Gausijani unutar *tile*-a moraju biti poređani po dubini. Umesto zasebnog sortiranja svakog *tile*-a, svi parovi (*Gausijan*, *tile*) sortiraju se odjednom po složenom ključu kod koga viši bitovi nose indeks *tile*-a a niži dubinu. Jedno takvo sortiranje istovremeno grupiše parove po *tile*-ovima i unutar svakog ih poređa *front-to-back* - tačno kako *alpha blending* i zahteva. Originalna implementacija to radi paralelnim *radix* sortiranjem na GPU-u; pošto sortiranje nije fokus ovog rada, ono se ovde izvršava na procesoru. Sortira se po dubini centra Gausijana, a ne po tačnoj dubini u svakom pikselu - standardna aproksimacija u 3DGS-u koja znatno pojednostavljuje sortiranje. Ona može izazvati treperenje (engl. *popping*) pri rotaciji kamere, što je poznato ograničenje izvornog algoritma [10], ali su za potrebe ovog rada te greške prihvatljive.

Redosled *front-to-back* omogućava i rano zaustavljanje. Kako transmitansa T iz (12) monotono opada, čim padne ispod malog praga preostali (dalji) Gausijani u *tile*-u mogu se preskočiti bez vidljive greške; na scenama koje imaju dosta neprovidnih objekata to znatno smanjuje broj stvarno obrađenih Gausijana po pikselu.

Za veličinu *tile*-a originalna implementacija postavlja na 16×16 piksela; ovaj rad koristi 32×32 iz hardverskih razloga.

Kompletan algoritam (*Slika 2*) tako čine učitavanje scene, tri pripremne faze (projekcija, dodela *tile*-ovima, sortiranje) i, na kraju, *alpha blending* koji proizvodi sliku. Pripremne faze organizuju podatke, dok *alpha blending* nosi najveći deo izračunavanja i predstavlja prirodnog kandidata za prebacivanje na čip.

Literatura

- [1] Ben Mildenhall et al. “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis”. In: *Proceedings of the European Conference on Computer Vision (ECCV)*. 2020.
- [2] Bernhard Kerbl et al. “3D Gaussian Splatting for Real-Time Radiance Field Rendering”. In: *ACM Transactions on Graphics* 42.4 (2023). DOI: 10.1145/3592433.
- [3] Jenny Lynn Almerol et al. “Accelerating Gravitational N -Body Simulations Using the RISC-V-Based Tenstorrent Wormhole”. In: *Proceedings of the SC ’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 2025. DOI: 10.1145/3731599.3767528.
- [4] Nick Brown, Jake Davies, and Felix LeClair. *Exploring Fast Fourier Transforms on the Tenstorrent Wormhole*. RISC-V for HPC Workshop, ISC High Performance. 2025. arXiv: 2506.15437 [cs.DC].
- [5] Maya Taylor et al. *Numerical Kernels on a Spatial Accelerator: A Study of Tenstorrent Wormhole*. 2026. arXiv: 2603.23343 [cs.PF].
- [6] Corentin Guezenoc. “Binaural Synthesis Individualization based on Listener Perceptual Feedback”. PhD thesis. CentraleSupélec, 2021. URL: <https://theses.hal.science/tel-03434565/>.
- [7] Matthias Zwicker et al. “EWA Splatting”. In: *IEEE Transactions on Visualization and Computer Graphics* 8.3 (2002), pp. 223–238. DOI: 10.1109/TVCG.2002.1021576.
- [8] Nelson Max. “Optical Models for Direct Volume Rendering”. In: *IEEE Transactions on Visualization and Computer Graphics* 1.2 (1995), pp. 99–108. DOI: 10.1109/2945.468400.
- [9] Thomas Porter and Tom Duff. “Compositing Digital Images”. In: *Proceedings of SIGGRAPH ’84, Computer Graphics*. Vol. 18. 3. 1984, pp. 253–259. DOI: 10.1145/800031.808606.
- [10] Lukas Radl et al. “StopThePop: Sorted Gaussian Splatting for View-Consistent Real-time Rendering”. In: *ACM Transactions on Graphics* 43.4 (2024). arXiv: 2402.00525.